

What is built, and what comes next

Three of eleven stages complete. This is a record of the features that **actually work today**, the stages still ahead, and the rules that hold across all of them.

Not an exam app with features bolted on.

An exam is one *mode* of a shared preparation engine, alongside practice, interviews and flashcard review. A preparation type — academic exam, interview, certification, coding test — is a row of configuration, never a subclass. That one decision is what lets this grow from “NIIT Semester 2” to “AWS Certification” without a rewrite.

STAGES DONE	BACKEND TESTS	MVP ENDS AT	SEED QUESTIONS
3 of 11	125	Stage 7	60

Features working today

Everything in this section is built, tested and running. Each entry says what it does and the one design decision that makes it more than a CRUD screen.

125	77	11	5
BACKEND TESTS	END-TO-END CHECKS	WEB ROUTES	QUESTION FORMATS

Accounts and sessions BUILT

Register, sign in, stay signed in across reloads, sign out everywhere.

- **Split-token design.** A short-lived token lives in browser memory only, so a script cannot lift it from storage. A long-lived refresh token sits in an httpOnly cookie, unreadable by script and revocable by the server.
- **Reuse detection.** Presenting a refresh token that has already been rotated means a copy escaped. Every session for that account is revoked rather than guessing which holder was legitimate.
- **Failures are indistinguishable.** A wrong password and an unknown account give the same answer, so the endpoint cannot be used to discover who has registered.

Preparation spaces BUILT

Create as many separate worlds as you need — a semester, an interview, a certification — each with its own material, history and progress.

- **Six preparation types** ship as configuration rows: academic exam, interview, certification, coding test, general test, custom. Each carries its own allowed question formats, session modes, scoring and difficulty policies, and readiness weights.
- **Per-space overrides.** A single space can adjust any of those settings without needing a preparation type of its own. Nested settings merge; lists replace, so a space that allows only one format means only that one.
- **Complete isolation.** Spaces share no data. Asking for someone else's space reports that it does not exist rather than that it is forbidden, so the API cannot be used to discover what exists.

Curriculum, with suggested topics BUILT

Subjects, and topics nested up to three levels — React, then Hooks, then useEffect. Subjects are the grain results are reported against.

- **Add a subject, get topics offered.** A shipped catalogue covers 17 subjects with six to ten topics each, proposed as chips you accept with one click.
- **Aliases resolve.** “js”, “html”, “react-native” and “postgres” all match, and the panel names the subject it matched so nothing happens silently.
- **Suggestions, never auto-insert.** Topics are what mastery and readiness are computed against, so a curriculum you did not choose would distort your own results. Accepted suggestions stop being offered; a *Suggest more* button keeps the rest available later.
- **Deliberately short lists.** A subject split into thirty pieces looks thorough, is miserable to work through, and leaves no single topic with enough attempts for its mastery score to mean anything.

Question bank BUILT

Write questions in five formats: multiple choice, multiple select, true/false, short answer and flashcards. Filter by subject, type, difficulty or text.

- **Questions carry no personal data.** No review schedule, no correctness counts. That separation is what keeps a bank shareable and lets the same question live in two spaces with entirely independent histories.
 - **Editing creates a version.** Fix a typo today and January's score still points at the exact text it was answered against.
 - **Answers are graded properly.** Multiple select penalises wrong selections, so ticking everything scores zero. Partial credit is scored but never counted as correct, which keeps accuracy honest. Short answer forgives case, accents, punctuation and a leading “the” — and stops there, because looser matching becomes unpredictable.
 - **Nothing is destroyed.** Archiving hides a question and frees its wording for reuse; the history it belongs to stays intact.
-

Import and export BUILT

Bring in material you already have as CSV or JSON, and take it back out again.

- **Dry run by default.** The whole file is understood before anything is written. You get a row-by-row report against spreadsheet line numbers, then decide.
 - **Duplicates caught both ways** — against the existing bank and within the file itself. Re-importing the same file adds nothing.
 - **Export is the same shape import accepts**, with subjects written by name rather than by id, so a bank moves between spaces and is never trapped in this app.
 - **60 seed questions** ship ready to import: 12 each across HTML, CSS, JavaScript, React and React Native.
-

The application itself BUILT

- **Dark and light**, applied before first paint so there is no flash.
- **Responsive**, and keyboard-navigable with a visible focus state throughout.
- **Errors say what to do.** Every failure returns a stable code and, where relevant, the specific fields at fault — the same shape whether it was a missing name or a multiple-choice question with two correct answers.

The eleven stages

No stage begins until the previous one builds, passes its tests and has been used by hand. Effort is measured in focused working sessions, not calendar time.

#	STAGE	WHAT IT DELIVERS	EFFORT	STATE
00	Scaffolding	Repo, build, database, both apps running	½	DONE
01	Foundation	Accounts, preparation spaces, curriculum	2-3	DONE
02	Question Engine	Question bank, versioning, import, export	2-3	DONE
03	Practice Sessions	The session engine and the immutable attempt log	2	NEXT
04	Exam Mode	Timed mocks, navigation, results breakdown	2	MVP
05	Adaptive Engine	Weak-topic detection, adaptive difficulty	3	MVP
06	Spaced Repetition	Review scheduling and flashcards	2	MVP
07	Analytics & Readiness	Dashboard and an explainable readiness score	2	MVP
08	AI & Interviews	Generation, explanation, interview follow-ups	3	LATER
09	PWA & Offline	Installable app, offline study, sync	2	LATER
10	Polish & Deploy	Accessibility, performance, security, hosting	2-3	LATER

03 · Practice Sessions NEXT

The single session engine every later mode reuses. Practice is simply the first mode implemented; exams, interviews and flashcard reviews are the same machinery with different rules. Building it once here is what stops three parallel copies of scoring, history and analytics.

- Answer submission, immediate feedback with the explanation, session summary.
- The **attempt log** — append-only, never updated. Every derived number in the system is rebuildable from it, which is what makes improving the algorithms later safe rather than destructive.
- **You verify:** run ten questions, get four wrong deliberately, check the summary and the recorded response times.

04 · Exam Mode

A mock under real conditions — and notably *not* a new subsystem: a session with a deadline, deferred feedback and a fixed blueprint. It deliberately does not adapt, because a mock that adjusts itself is not a yardstick you can compare across weeks.

- Configurable count and duration, navigation grid, mark-for-review, auto-submit.
- Results broken down by subject, topic and difficulty, with time per question.
- **The timer is server-authoritative**, or your own scores would mean nothing.

05 · Adaptive Engine

The part that makes this not a quiz app. It learns what you are weak at and serves you that, pitched to stretch you without breaking you.

- An ability estimate per topic and a matching difficulty rating per question, both updating after every answer — so a question you mislabelled corrects itself.
 - **Targets roughly 78% success:** high enough to stay motivating, low enough to meet real resistance. Difficulty is relative to you, so a “hard” question quietly becomes a medium one as you improve.
 - **Guardrails are a feature.** A recovery mode after a bad run, a guaranteed win every few questions, and a topic is never called *weak* on fewer than five attempts — only *unassessed*.
-

06 · Spaced Repetition & Flashcards

Deciding *when* something should come back, so you revise just as you are about to forget. A flashcard here is a question type, not a separate system — which is why Again/Hard/Good/Easy feeds the same engine that drives exam readiness, for free.

- Ordinary quiz answers are translated into a grade using correctness, response time and confidence, so a lucky guess is not scheduled as solid knowledge.
 - Interval previews on each button, before you press it.
-

07 · Analytics & Readiness MVP ENDS HERE

The daily answer to “what should I do now, and am I actually ready?” Readiness is deliberately not your average quiz score: six weighted components covering coverage, accuracy, depth, retention, consistency and mock performance.

- Every snapshot stores its own breakdown, so the app can say “*readiness fell three points because fourteen items are overdue — clearing them is worth about four*” rather than showing a number that cannot explain itself.
 - **Scaled by how much evidence exists.** After twelve questions you cannot be “88% ready”, and a system that says so is lying at the moment the stakes are highest.
-

08–10 · AI, Offline, Polish

- **AI is an enhancement, never a foundation.** With no provider configured the application stays fully functional and every test still passes. Nothing generated reaches your bank without a review step.
- **Offline** comes late because it mirrors a data model that must have stopped changing. Attempts are append-only events with a client id, so syncing is conflict-free by construction.
- **Polish** is accessibility, performance, a security audit, end-to-end tests, backup and deployment.

The rules that hold across every stage

Not aspirations in a document nobody re-reads. Most are enforced by tests that fail the build, or by constraints the database itself refuses to violate.

- **An exam is a mode, not an entity.** One session engine serves practice, exams, interviews and flashcards. The moment an exam becomes a first-class thing, everything cross-cutting gets written three times.
- **A preparation type is a row, not a subclass.** Adding “PMP Certification” is an insert. A genuinely new format costs one small strategy class, not a parallel copy of the session, scoring and analytics stack.
- **Questions carry no performance data.** Review state is keyed by user, space and question, so the same imported question lives in two spaces with independent histories.
- **The attempt log is append-only.** Every derived number can be thrown away and rebuilt from it, which is what makes improving the algorithms safe.
- **Nothing is scoped by accident.** The space identifier is on every per-space table and required by every query. Cross-space references are rejected by the database, not merely discouraged in review.
- **Access failures look like absence.** Requesting someone else's data returns “not found”, never “forbidden”, so the API cannot be used to discover what exists.
- **The engine is testable without a database.** Selection and scheduling take plain values in and return plain values out. A build-failing rule enforced this from before the first line of that engine was written.
- **Nothing is deleted that history references.** Archiving hides; it never destroys months of your own study record.
- **Secrets live in the environment.** No key or connection string in the source, and the application refuses to start with a placeholder signing key.
- **Your material is never trapped.** Export produces exactly what import accepts.

The governing requirement

Build the foundation so this can evolve from an exam preparation tool into a general-purpose adaptive preparation platform **without requiring a rewrite**. Every decision above was chosen against that single test.